

Publishing spec — the human approval gate (Lab 10)

Multi AI Agents Workflow for Content Creation (Claude Edition) · Day 2 · Use Claude Code

Nothing reaches LinkedIn, Facebook or YouTube unless a **person** approved

that exact version.

Content files

One file per item: content/<channel>/<week-or-issue>/<id>.md

```
---
id: li-w01-01
channel: linkedin          # linkedin | facebook | youtube | newsletter
title: Know your number in 30 minutes
link: https://tertiarycourses.github.io/TGS-2023036153-Multi-AI-Agents-Workflow-for-Content-
Creation/?utm_source=linkedin&utm_medium=social&utm_campaign=know-your-number
video: content/video/ep01/ep01.mp4  # youtube only
tags: retirement, cpf, planning      # youtube only
---
The post text (for YouTube: the video description).
```

review/approvals.csv

id,channel,file,sha256,status,submitted_by,approver,approved_at,notes

- status: pending | approved | changes | rejected
- sha256: hash of the content file (plus the video file for YouTube).

Scripts (Node 22, no packages)

Script	Who runs it	What it does
scripts/submit.mjs <file>	agents	Adds or refreshes a pending row with the file's hash
typing "approved <id> by <name>" (or "changes <id>: why") in Claude Code	a person	Approves (or sends back) the CURRENT version: records approver, time and hash. A hook reads the person's own message, so an agent cannot approve.
scripts/publish.mjs <id> [--live]	agents or people	Dry run by default: prints the exact request, sends nothing. With --live: refuses unless the row is approved AND the hash still matches; refuses an id already in review/publish-log.csv (no double posts); logs the post id.

The hook (Claude Code, .claude/settings.json)

A PreToolUse hook on Bash, Edit and Write runs scripts/gate-hook.mjs:

- blocks any agent that tries to approve (only people approve, by typing in the chat);
- blocks agent edits to review/approvals.csv;
- blocks publish.mjs ... --live unless the item is approved and unchanged.

It blocks with exit code 2 and a reason the agent can read.

The APIs

LinkedIn — POST <https://api.linkedin.com/rest/posts>

Headers: Authorization: Bearer <token>, LinkedIn-Version: <YYYYMM>,

X-Restli-Protocol-Version: 2.0.0, Content-Type: application/json.

Body: author (urn:li:person:...), commentary, visibility PUBLIC,

distribution {feedDistribution MAIN_FEED}, lifecycleState PUBLISHED.

Needs the w_member_social permission. The post id comes back in the

x-restli-id header.

Facebook Page — POST <https://graph.facebook.com/<version>/<page-id>/feed>

with message, link and a Page access token (pages_manage_posts,

pages_read_engagement).

YouTube — resumable upload:

POST <https://www.googleapis.com/upload/youtube/v3/videos?uploadType=resumable&part=snippet,status>

with snippet (title, description, tags, categoryId 27) and status

(privacyStatus **private**, selfDeclaredMadeForKids false), then PUT the

video bytes to the returned Location. Get an access token from the refresh

token at <https://oauth2.googleapis.com/token>. Uploads from an unaudited API

project stay private — a person makes the video public in YouTube Studio.

Newsletter — no API. Created as files and Gmail drafts (Lab 11).